

Salesforce Day 3

Automation with Flows & Validation Rules – Daily Task Documentation

Name:	Golla Vaishnavi
Regd No:	23PA1A0572

Topic: Automation using Validation Rules, Flows, and Triggers

Today's task focused on Salesforce declarative and programmatic automation for the Placement Management System. The session covered the order of execution between Validation Rules, Flows, and Triggers, and worked through a real business scenario for the Placement Cell that needed additional automation such as duplicate-application prevention, automatic date population, CGPA-based rejection, email notifications, and Offer Letter creation when a student's status changes to Selected.

Learning Objectives Covered

- Understood whether a Validation Rule can update another field, and why it cannot.
- Understood the order of execution between Validation Rule, Flow, and Trigger.
- Learned what a Record-Triggered Flow is and when to use it.
- Mapped real business requirements to the correct automation tool: Validation Rule, Flow, or Trigger.
- Built a Record-Triggered Flow with a Decision element and multiple Update Records actions.
- Understood how Validation Rules block invalid record saves and surface error messages to the user.
- Compared Flow versus Trigger for different automation situations (field updates, related record creation, email, REST callouts, bulk processing).

Work Completed

1. checkingstatus Record-Triggered Flow

Built a Record-Triggered Flow named **checkingstatus** on the **Appointment** object, configured to run whenever a record is created or updated, optimized for Fast Field Updates. The flow starts with a Decision element, **checkingwaitingstatus**, which branches into two outcomes: **isavailable** and **notavailable**.

2. Update Records Actions

For the **isavailable** path, added an Update Records action, **WaitingAvailable**. For the **notavailable** path, added a second Update Records action, **WaitingnotAvailable**. Both paths merge and terminate at the End element, demonstrating how a Decision element can drive different field updates depending on record data — the same pattern used for Application status automation.

3. Flow Activation and Testing

Saved the flow as an active version and confirmed it shows an **Active** status in Flow Builder, ready to fire automatically on record creation or update without any manual intervention.

4. Validation Rule Enforcement

Tested data-quality enforcement by creating a New Contact record. Attempting to save the record with incomplete required information triggered a validation error, "We hit a snag – cannot be inserted," confirming that a Validation Rule correctly blocks the record from being saved until the data meets the

required conditions — the same approach used for enforcing CGPA and mandatory-field rules in the Placement Management System.

Requirement-to-Automation Mapping

Based on the business scenario, each automation requirement was mapped to the most appropriate Salesforce tool as follows:

Requirement	Best Tool	Reasoning
Reject duplicate applications	Validation Rule	A simple record-level condition check before save; no related records need
Auto-fill Application Date	Flow	A Record-Triggered Flow can set a field value automatically on create with
Send Email	Flow	Flow has a built-in Send Email action, avoiding the need for Apex email co
Reject low CGPA	Validation Rule	A formula comparing CGPA to the minimum requirement blocks the save o
Create Offer Letter record	Flow (or Trigger for complex logic)	Flow can create a related record declaratively; a Trigger is preferred only if

Key Learnings

- A Validation Rule cannot update another field — it can only evaluate a condition and either allow or block the save with an error message; field updates require a Flow, Trigger, or Workflow/Process Builder instead.
- Order of execution: System validation rules run first, followed by Before-Save Flows and Before Triggers, then custom Validation Rules, then After-Save Flows and After Triggers.
- A Record-Triggered Flow runs automatically when a record is created, updated, or deleted, and can branch its logic using Decision elements to update different fields conditionally.
- Declarative tools (Validation Rules, Flows) are preferred for straightforward field updates, related record creation, and email notifications, while Apex Triggers are best reserved for complex multi-object calculations, external REST callouts, and heavy bulk processing.
- Running the same field update from multiple automations (Trigger, Flow, and Workflow) on the same object can cause conflicting or repeated executions, so automation logic should be consolidated into a single, well-designed process wherever possible.

Screenshots

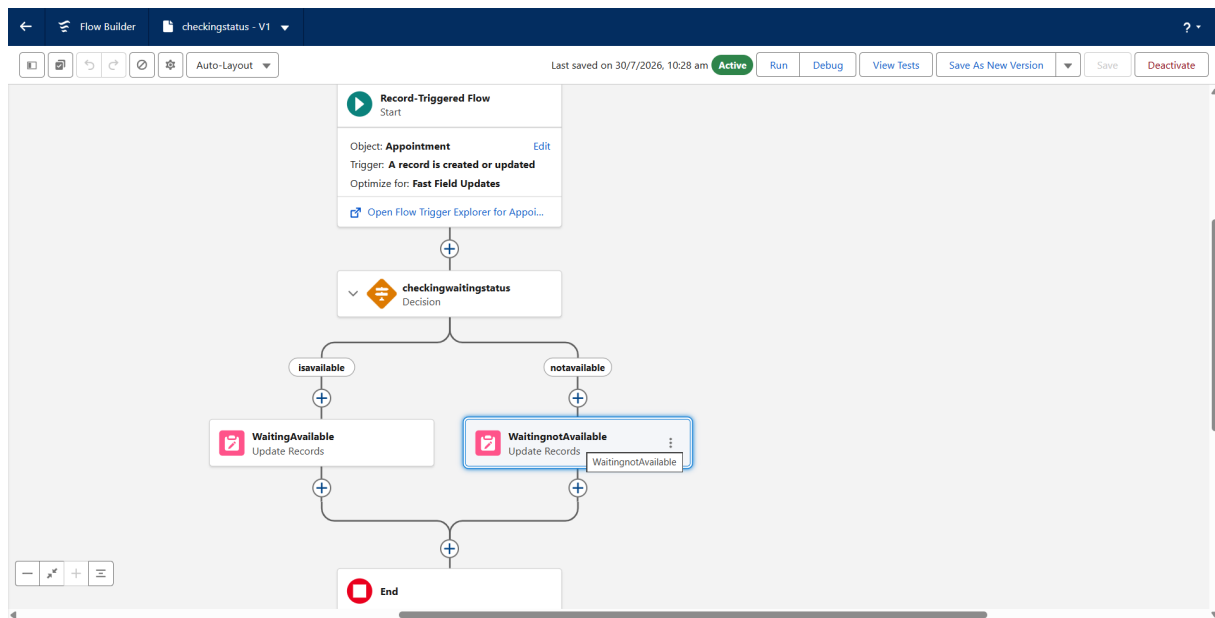


Fig 1: checkingstatus Record-Triggered Flow with the checkingwaitingstatus Decision element and its two Update Records branches.

New Contact

* = Required Information

Contact Information

Contact Owner
GOLLA VAISHNAVI

*** Name**

Salutation
Mr.

First Name
GOLLA

*** Last Name**
VAISHNAVI

Account Name
vaishnavi

Title

Department

Birthdate

Phone

Home Phone

Mobile

Other Phone

Email
23pa1a057...

⚠ We hit a snag.

Review the errors on this page.

- cannot be inserted

Cancel

Save & New

Save

Fig 2: New Contact form showing a Validation Rule error, "We hit a snag – cannot be inserted," when required data is missing.

README

What was built today?

A Record-Triggered Flow (checkingstatus) on the Appointment object that checks waiting status through a Decision element and updates records down two branches (Available / Not Available), plus a Validation Rule tested on the Contact object to enforce required data before a record can be saved.

Which object does the Flow run on?

The Appointment object, triggered when a record is created or updated, optimized for Fast Field Updates.

What does the Decision element do?

It evaluates the waiting status of the record and routes execution down the 'isavailable' or 'notavailable' path, each leading to a different Update Records action.

What does the Validation Rule demonstrate?

That Salesforce blocks a record from being saved and shows an on-screen error ("cannot be inserted") when a required condition is not met, rather than silently allowing bad data into the system.

What was learned today?

The difference in responsibility between Validation Rules, Flows, and Triggers; how to map a business requirement to the right automation tool; and how to build and activate a Record-Triggered Flow with branching logic.